

Smart Expense Tracker

Java Programming Build Your Own Project Flipped
Course Evaluation

Name: Kavya Sharma
Reg. No.:25BAI11070
Branch: CSE AI and ML
Course code: CSE2006

1. Introduction

Smart Expense Tracker is a Java-based console application used to record and manage daily expenses. It helps users calculate spending, search expenses, and monitor their monthly budget.

2. Problem Statement

Managing expenses manually can make it difficult to track total spending and control a monthly budget. This project provides a simple system to record and analyze expenses.

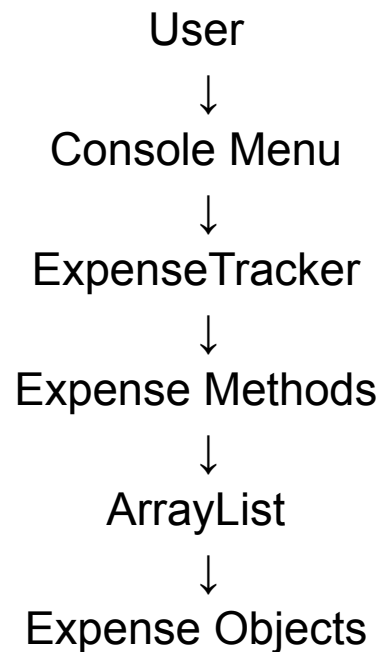
3. Functional Requirements

- Add expense
- View all expenses
- Search by category
- Calculate total expenses
- Find highest expense
- Set monthly budget
- Check budget status
- Delete expense
- Show expense summary

4. Non-Functional Requirements

- Simple and user-friendly
- Fast response
- Reliable calculations
- Easy to maintain
- Runs on systems with Java JDK

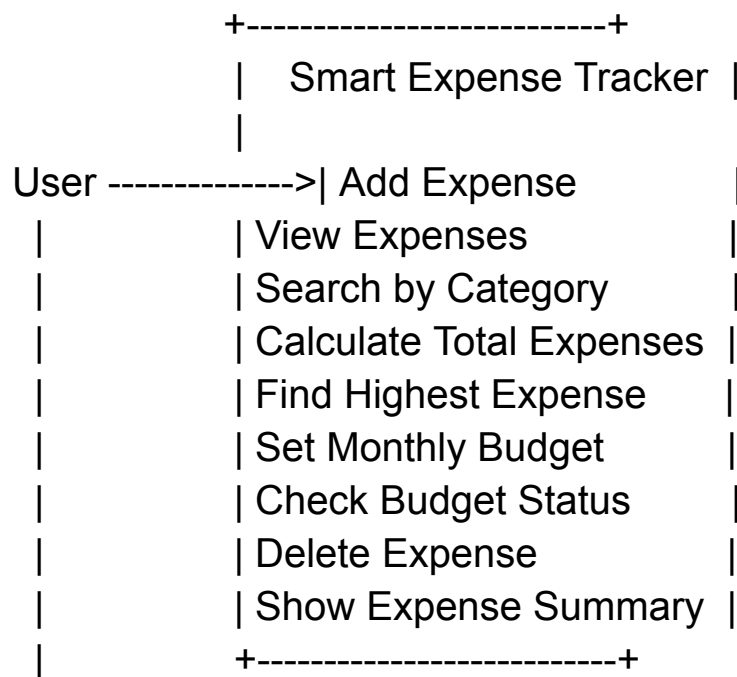
5. System Architecture



6. Design Diagrams

6.1 Use Case Diagram

The Use Case Diagram shows how the user interacts with the Smart Expense Tracker.

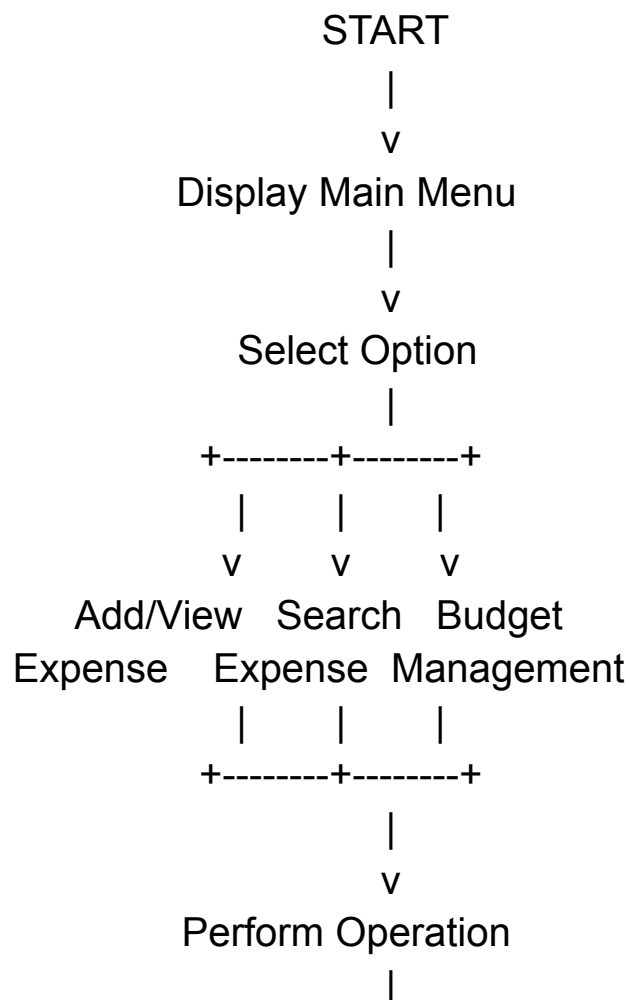


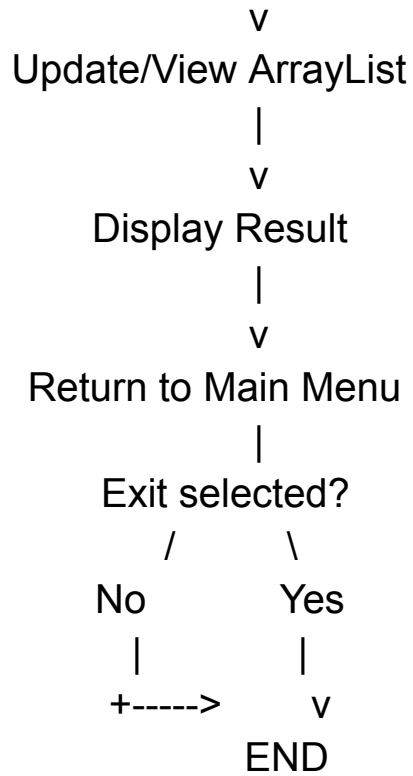
Actor: User is the main actor who interacts with all the features of the application.

Main Use Cases:

- Add expense
- View expenses
- Search expenses
- Calculate total
- Find highest expense
- Set budget
- Check budget
- Delete expense
- View summary

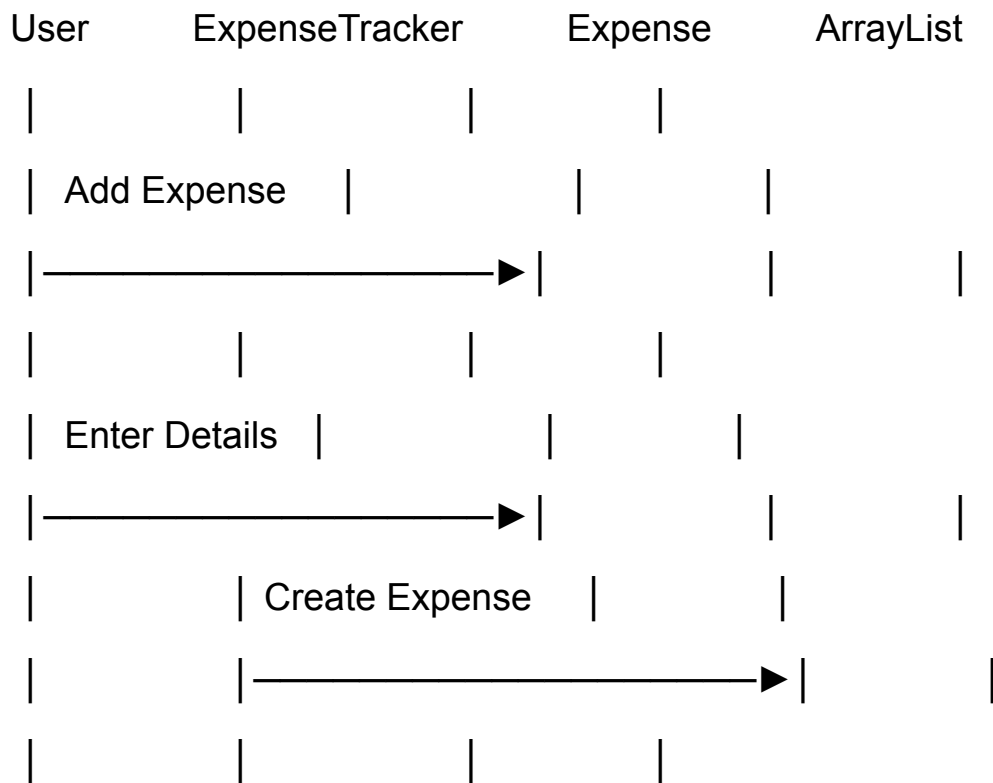
6.2 Workflow Diagram

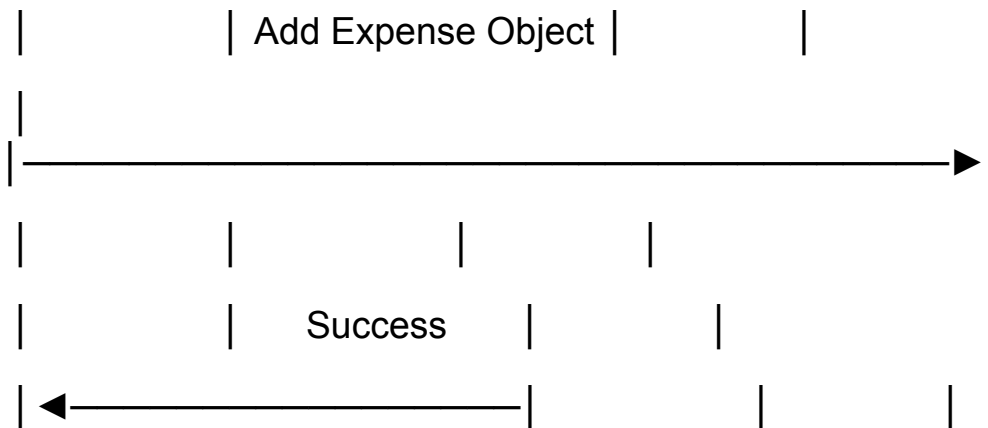




6.3 Sequence Diagram

Example: **Adding an Expense**

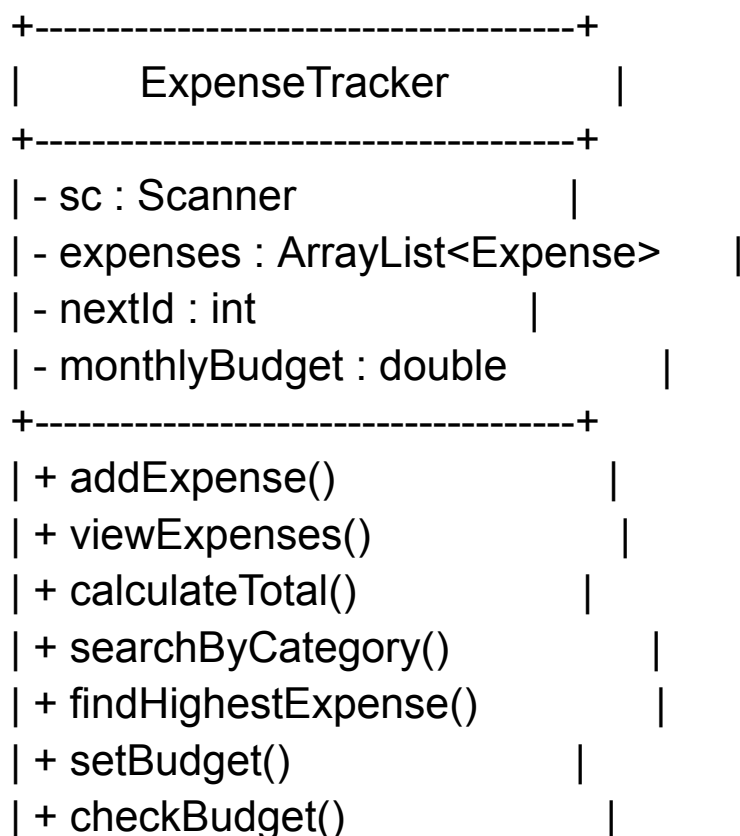


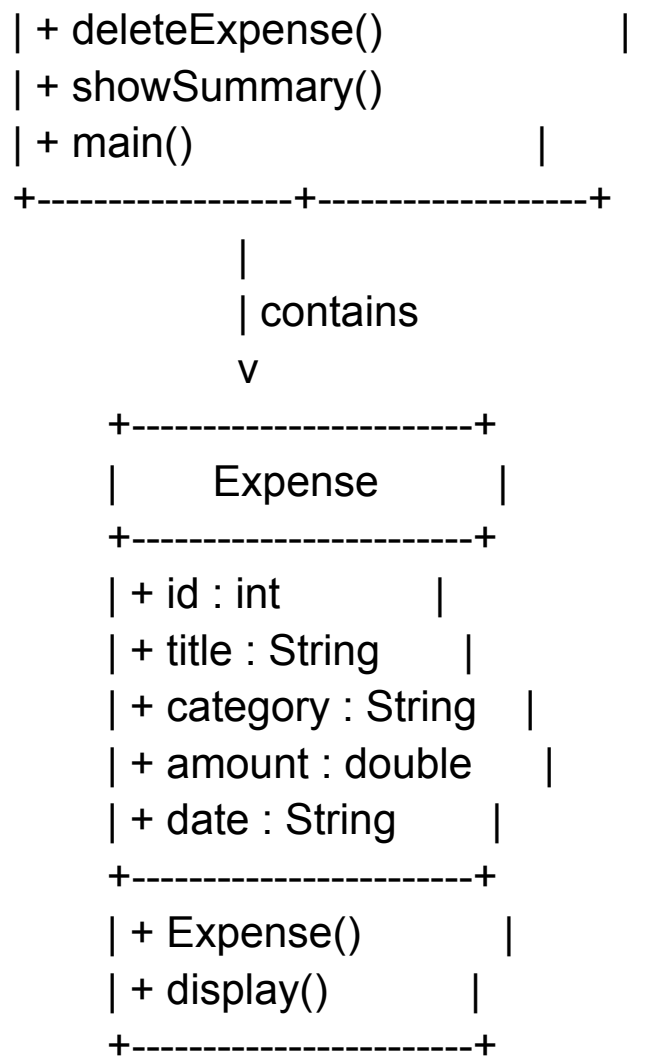


Flow:

1. User selects **Add Expense**.
2. System accepts expense details.
3. An **Expense** object is created.
4. The object is stored in the **ArrayList**.
5. A success message is displayed.

6.4 Class/Component Diagram





Relationship: `ExpenseTracker` manages multiple `Expense` objects using an `ArrayList`.

7. Design Decisions & Rationale

- **Java Console Application:** Simple and easy to use.
- **OOP Design:** `Expense` class organizes expense data.
- **ArrayList:** Stores multiple expenses dynamically.
- **Menu-Driven:** Makes different operations easy to access.
- **Separate Methods:** Keeps the code modular and readable.
- **Unique ID:** Helps identify and delete expenses.

- **No Database:** Current version uses temporary ArrayList storage.
- **Budget Feature:** Helps track spending against the monthly budget.

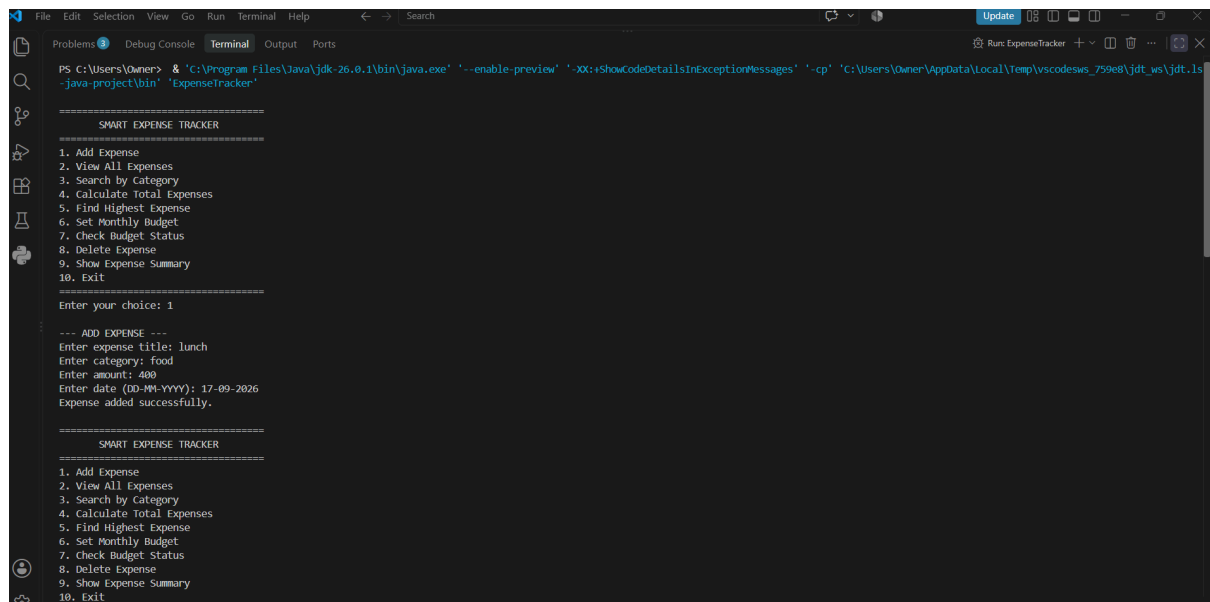
8.Implementation

The project contains two main classes:

Expense: Stores ID, title, category, amount, and date.

ExpenseTracker: Handles adding, viewing, searching, deleting, calculating expenses, budget management, and summary generation.

9.Screenshots / Results



```
PS C:\Users\Owner> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Owner\AppData\Local\Temp\vscodesws_759e8\jdt_ws\jdt.ls' '-java-project\bin' 'ExpenseTracker'
```

```
=====
SMART EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Calculate Total Expenses
5. Find Highest Expense
6. Set Monthly Budget
7. Check Budget Status
8. Delete Expense
9. Show Expense Summary
10. Exit
=====
Enter your choice: 1

--- ADD EXPENSE ---
Enter expense title: lunch
Enter category: food
Enter amount: 400
Enter date (DD-MM-YYYY): 17-09-2026
Expense added successfully.

=====
SMART EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Calculate Total Expenses
5. Find Highest Expense
6. Set Monthly Budget
7. Check Budget Status
8. Delete Expense
9. Show Expense Summary
10. Exit
```

```
File Edit Selection View Go Run Terminal Help
Problems Debug Console Terminal Output Ports
10. Exit
=====
Enter your choice: 1

--- ADD EXPENSE ---
Enter expense title: Dinner
Enter category: food
Enter amount: 500
Enter date (DD-MM-YYYY): 18-09-2026
Expense added successfully.

=====
SMART EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Calculate Total Expenses
5. Find Highest Expense
6. Set Monthly Budget
7. Check Budget Status
8. Delete Expense
9. Show Expense Summary
10. Exit
=====
Enter your choice: 9

===== EXPENSE SUMMARY =====
Number of Expenses : 2
Total Expenses      : 900.00
Average Expense     : 450.00
=====

SMART EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
```

```
File Edit Selection View Go Run Terminal Help
Problems Debug Console Terminal Output Ports
7. Check Budget Status
8. Delete Expense
9. Show Expense Summary
10. Exit
=====
Enter your choice: 9

===== EXPENSE SUMMARY =====
Number of Expenses : 2
Total Expenses      : 900.00
Average Expense     : 450.00
=====

SMART EXPENSE TRACKER
=====
1. Add Expense
2. View All Expenses
3. Search by Category
4. Calculate Total Expenses
5. Find Highest Expense
6. Set Monthly Budget
7. Check Budget Status
8. Delete Expense
9. Show Expense Summary
10. Exit
=====
Enter your choice:
PS C:\Users\Owner>
```

10. Testing

The application was tested by:

- Adding different expenses
- Viewing expense records
- Searching categories
- Calculating total and average
- Setting and checking budget
- Finding the highest expense
- Deleting an expense

11. Challenges

- Handling user input correctly
- Managing multiple expense objects
- Calculating totals and averages
- Finding the highest expense
- Implementing budget calculations
- Managing the menu using loops and switch-case

12. Learnings & Key Takeaways

During the development of the Smart Expense Tracker, the following concepts and skills were learned:

- Understanding of **Object-Oriented Programming (OOP)** in Java.
- Learned how to create and use **classes, objects, and constructors**.
- Used **ArrayList** for storing multiple expense records.
- Improved understanding of **methods and modular programming**.

- Practiced **if-else, switch-case, loops, and conditional statements**.
- Learned how to perform **searching, calculation, and deletion operations**.
- Understood the importance of **unique IDs** for managing records.
- Learned how to design a **menu-driven console application**.
- Improved debugging and problem-solving skills.
- Learned how to organize a Java project in a simple and maintainable way.

13. Future Enhancements

The Smart Expense Tracker can be improved in the future by adding:

1. **Database Integration**

Use MySQL and JDBC to permanently store expense records.

2. **User Login System**

Add registration and login functionality for multiple users.

3. **Income Management**

Add income records and calculate savings.

4. **Advanced Reports**

Generate daily, monthly, and yearly expense reports.

5. **Graphical Interface**

Convert the console application into a GUI using Java Swing or JavaFX.

6. **Expense Charts**

Display expenses using pie charts and bar graphs.

7. **Export Functionality**

Allow users to export expense reports to CSV or PDF.

8. Budget Notifications

Notify users when their expenses approach or exceed the monthly budget.

14. References

1. Oracle Java Documentation – Java Language and API Documentation.
2. Oracle – Java Tutorials and Learning Resources.
3. Java Documentation – `ArrayList`, `Scanner`, Classes and Methods.
4. Herbert Schildt, *Java: The Complete Reference*.
5. VIT Bhopal University course materials and laboratory notes.